

2048 Master How-To Guide

CMPT 310, Group 1

Introduction

2048 Master is a project that builds an AI agent to play 2048, the sliding-tile puzzle game. The game is played on a 4x4 grid of tiles. Each turn, the agent slides all tiles in one of four directions (up, down, left, right); when two tiles with the same value collide, they merge into a single tile with twice the value, and the merged tile's value is added to the score. A move is considered legal if applying it changes the board, meaning tiles slide or merge. After every legal move, a random tile with value 2 or 4 is spawned. The game ends when no legal move remains.

2048 is a reinforcement learning problem: the agent receives the board state as input and produces one of the four moves as output. The outcome of a move is not fully under the agent's control: the random spawning of tiles makes the environment stochastic, so the same move from the same board can lead to different results. The reward is the score gained from merges, and the agent's goal is to learn to maximize the score it can reach over a game.

This guide explains how we built the agent, step by step. We first implement the game environment (board representation, movement and merging, tile spawning, and rewards). We then build an N-tuple network that estimates how good a board position is, and train it with temporal difference (TD) learning through self-play. We use the learned value function as the evaluator of an Expectimax search, and compare the final agent against a random baseline. The guide also covers the Pygame interface and common troubleshooting issues.

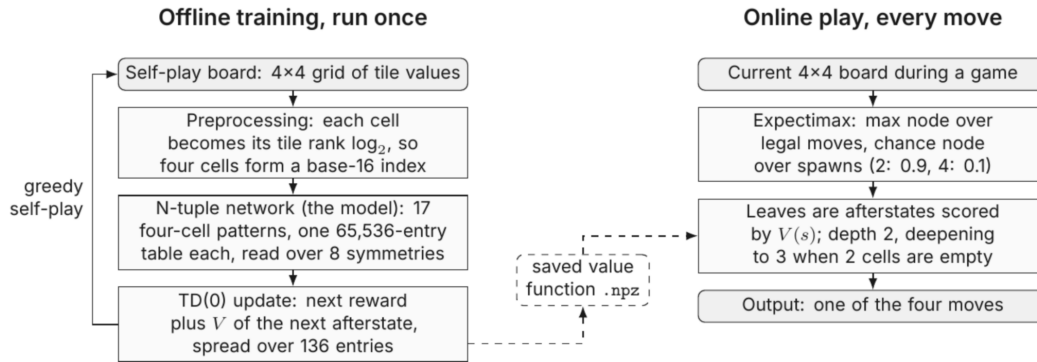


Figure 1. The 2048 Master reinforcement learning pipeline.

Setting Up the 2048 Environment

Before implementing the AI agents, we first built a game environment that represents the rules of 2048 and provides a consistent interface for agents to interact with the game.

1. Board Representation

We represent the 2048 board as a 4x4 NumPy array. Each cell stores the numerical value of the tile at that position, while an empty cell is represented by 0. For example:

[0, 2, 0, 0] [0, 0, 0, 0] [0, 0, 4, 0] [2, 0, 0, 0]

This representation allows the environment to efficiently perform board operations and provides the current game state directly to the AI agents.

2. Movement and Merging

A single move in 2048 consists of two operations: sliding all the tiles toward the selected direction and merging adjacent tiles with the same value. For example, for a left move, we implement the following sequence:

1. Remove all empty cells from the row by sliding non-zero tiles to the left.

2. Merge adjacent tiles if they have the same value.
3. Slide the row again to remove the empty spaces created by merging.

For a single row, this is represented as:

Before: [2, 2, 4, 0] Merge: [4, 0, 4, 0] Slide: [4, 4, 0, 0]

Rather than implementing four independent movement algorithms, we reuse the same left-movement logic through board transformations:

- Left: apply the left-movement logic directly.
- Right: reverse each row, apply left movement, then reverse it back.
- Up: transpose the board, apply left movement, then transpose it back.
- Down: transpose the board, apply right movement, then transpose it back.

This reduces duplicated code and keeps the merging behavior consistent across all directions.

```
# Performs a full LEFT move on a single row (slide + merge + slide)
def move_row_left(row):
    # Step 1: slide
    slid_row = slide_row_left(row)

    # Step 2: merge
    merged_row, merged_values, merged_changed = merge_row_left(slid_row)

    # Step 3: slide again (to remove gaps after merge)
    final_row = slide_row_left(merged_row)

    # Check if any changes occurred (either from sliding or merging)
    changed = not np.array_equal(row, final_row) or merged_changed

    return final_row, merged_values, changed

# Performs a full LEFT move on the entire board
# It applies row-based movement to every row
def move_left(board):
    new_board = np.zeros_like(board)
    all_merged_values = []
    changed = False

    for i in range(board.shape[0]):
        new_row, merged_values, row_changed = move_row_left(board[i])

        new_board[i] = new_row
        all_merged_values.extend(merged_values)

        if row_changed:
            changed = True

    return new_board, all_merged_values, changed
```

Figure 2. Code excerpt implementing left movement and merging.

3. Random Tile Spawning

At the beginning of a game, two tiles are placed on randomly selected empty cells. After each legal move, the environment randomly selects an empty cell and places either a 2, with 90% probability, or a 4, with 10% probability. The movement functions return a **changed** flag indicating whether the move modified the board, so a new tile is spawned only when this flag is true. This random spawning makes 2048 a stochastic environment: the agent controls its actions, but it cannot control where or which value the next tile will appear.

4. Rewards

The environment uses the score increase from tile merges as the immediate reward. When two equal tiles merge, the resulting tile’s value is added to the score. For example, $[4, 4] \rightarrow [8]$, so the reward is 8.

A move that only rearranges tiles without creating a merge receives a reward of 0. If multiple merges occur during one move, their values are added together to produce the total reward for that transition. This reward design directly connects the agent’s actions to the objective of achieving higher-value tiles.

Building the N-Tuple Value Function

The RL agent needs a value function $V(s)$ that estimates how good a board position is. We represent it with an N-tuple network: a set of lookup tables that evaluate a board by summing selected entries. We chose this approach over a neural network because 2048 has a vast state space. An N-tuple network captures useful local board structure with compact tables that can be trained efficiently using TD learning.

Tile ranks. The first step is to change how each tile is represented. Every tile in 2048 is a power of two (2, 4,

8, 16, ...), so instead of storing the value itself, we store its exponent: an empty cell becomes 0, and the tiles 2, 4 and 8 become 1, 2 and 3. This gives each cell one of 16 possible symbols, making the representation suitable for a finite lookup table. Four cells can then be described by a number with four digits, which is exactly what a table index needs.

Tuple patterns. A tuple is a fixed pattern of four cells. We use 17 tuples: the four rows, the four columns, and the nine overlapping 2x2 squares (Figure 1). Because merges happen between adjacent tiles along rows and columns, these patterns capture the local interactions that are important for evaluating a board.

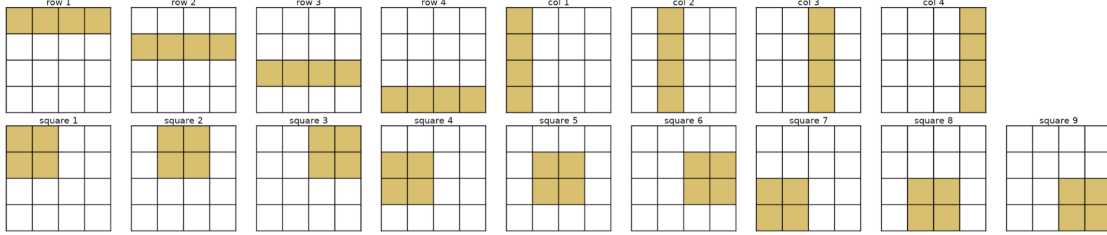


Figure 3. The 17 four-cell tuple patterns.

Lookup tables. Each tuple pattern has its own lookup table, with one slot for every possible combination of values its four cells can contain. With 16 possible values per cell, each table contains $16^4 = 65,536$ slots. When the network evaluates a board, it reads the slot selected by each pattern and sums the values it finds. Initially, all slots are zero, so the network has no learned value for any position and training gradually updates these entries.

Base-16 encoding. To select an entry from a table, the four values in each tuple must be converted into a single index. We encode the four cell values as the digits of a base-16 number. The first cell contributes the units digit, while the remaining cells contribute multiples of 16, 256 and 4096. For example, the tiles [8, 4, 2, 2] have exponents [3, 2, 1, 1], which encode as $3 + 2(16) + 1(256) + 1(4096) = 4387$, so this combination of tiles is stored at index 4387 in that pattern’s table. The following code implements this encoding:

```
def encode(tile_values):
    ranks = tile_ranks(tile_values)          # tiles -> exponents, e.g.
    [8,4,2,2] -> [3,2,1,1]
    return ranks[0] + 16 * ranks[1] + 256 * ranks[2] + 4096 * ranks[3]
```

Figure 4. Base-16 tuple encoding in the N-tuple network.

Symmetry handling. Rotated and reflected versions of the same board are strategically equivalent as the same moves are available and they lead to the same outcomes. Therefore, each board has eight equivalent orientations: four rotations and four rotations of a reflected version. We exploit this by evaluating each board across all eight orientations using the same lookup tables. With 17 patterns and 8 orientations, each board contributes to 136 table entries. This allows each training position to provide information across all of its equivalent orientations. During training, the TD correction is divided evenly across these entries. We use `np.add.at` so that entries selected more than once are updated correctly.

```
def value(self, grid: np.ndarray, validate: bool = True) -> float:
    indices = self.feature_indices(grid, validate=validate)
    table_numbers = np.arange(len(self.tuples), dtype=np.intp)[None, :]
    return float(self.tables[table_numbers,
                             indices].sum(dtype=np.float64))

def update(self, grid: np.ndarray, delta: float) -> None:
    """Distribute delta evenly over every table lookup used by value()."""
    try:
        numeric_delta = float(delta)
    except (TypeError, ValueError) as error:
        raise TypeError("delta must be a finite real number") from error
    if not np.isfinite(numeric_delta):
        raise ValueError("delta must be finite")

    indices = self.feature_indices(grid, validate=False)
    amount_per_lookup = numeric_delta / indices.size
    offsets = np.arange(len(self.tuples), dtype=np.int64) * TABLE_SIZE
    np.add.at(self.tables.reshape(-1), (indices + offsets).reshape(-1),
              amount_per_lookup)
```

Figure 5. Symmetry-aware evaluation and duplicate-safe updates.

Training with TD Learning

We use TD learning to train the N-tuple network value function through self-play. The N-tuple network’s lookup tables are initially set to zero. As the agent plays games, TD updates adjust these values based on the rewards and future afterstates it encounters. The goal is for the value function to assign higher values to board positions that are more likely to lead to high scores and longer games.

1. Afterstates

A key part of our TD learning implementation is the use of afterstates. An afterstate is the board immediately after the player’s move has been applied, but before the random tile is spawned. This distinction is useful because a move in 2048 is deterministic. We train the value function on these afterstates because they represent the direct result of the agent’s decision before the randomness introduced by the environment. This allows the agent to evaluate the resulting board for each legal action directly, rather than learning a separate value for each action.

2. TD Update

For each possible action, the trainer evaluates the resulting afterstate by combining the immediate merge reward with the learned value of that afterstate. The action with the highest value is selected:

$$\begin{aligned}Q(s, a) &= r(s, a) + V(s'_a), \\ \text{target} &= r_{t+1} + V(s'_{t+1}), \\ \delta_t &= \text{target} - V(s'_t), \\ V(s'_t) &\leftarrow V(s'_t) + \alpha \delta_t.\end{aligned}$$

In our implementation, the value function estimates future rewards from an afterstate, so the immediate reward is handled separately during action selection. The terminal state has no future reward, so its TD target is set to zero.

3. Self-Play Training Loop

The agent does not require an external dataset. Instead, it generates training data by repeatedly playing 2048 games against itself. Each complete game is treated as one training episode. For each episode:

1. **Initialize the game:** Start a new board and spawn the initial two tiles.
2. **Generate legal actions:** Test the four possible moves and discard actions that do not change the board.
3. **Evaluate afterstates:** Apply each legal action without spawning a tile and calculate its immediate reward plus N-tuple value.
4. **Select and execute an action:** Choose the action with the highest estimated value. The environment executes the move, calculates the reward, and spawns a random tile.
5. **Update the value function:** Evaluate the best afterstate from the resulting board, calculate the TD target and error, and update the previous afterstate’s N-tuple values.
6. **Repeat:** Continue until no legal moves remain, then start the next episode.

```
next_action, next_afterstate, _next_reward, target = self.evaluate_moves(
    game.board.grid
)
if next_action is None: # terminal: no future reward
    target = 0.0

delta = target - self.network.value(afterstate, validate=False)
self.network.update(afterstate, self.alpha * delta)
```

Figure 6. The main TD self-play update.

4. Training Configuration

An episode is one complete game of 2048. We trained the agent for 10,000 episodes to expose the value function to a wide range of board configurations. We experimented with different learning rates during development and selected the final value based on training performance and stability. Checkpoints are periodically saved as `.npz` files containing the current N-tuple lookup tables. These checkpoints allow the trained value function to be evaluated or training to be continued without starting from zero.

```
class TDTrainer:
    def __init__(
        self,
        network: NTupleNetwork,
        learning_rate=0.01,
        episodes=10000, # 10,000 games
        log_every=500, # print + checkpoint interval
        checkpoint_path="checkpoints/value_function.npz",
        log_path="training_logs/training_scores.csv",
    ):

```

Figure 7. The training configuration excerpt.

5. Monitoring Training Progress

To determine whether the agent is learning, we record the score from each training episode and calculate the average score. An increasing average score indicates that the agent is generally learning to produce better game outcomes over time.

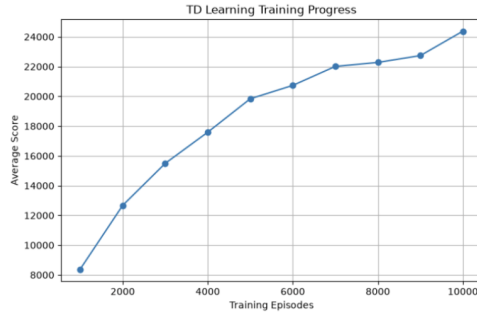


Figure 8. Average score during 10,000 self-play episodes.

Integrating the Learned Value Function with Expectimax

Expectimax needs two things to make decisions: a model of the game’s randomness and a way to evaluate board positions. The chance nodes model the random spawning of a 2 or 4 tile, while the evaluation function estimates how promising a board position is. For our original Expectimax agent, this evaluation was a hand-designed heuristic. For the RL agent, we replaced that heuristic with the value function learned by the N-tuple network through TD self-play.

The search alternates between max nodes, where the agent chooses the best legal move, and chance nodes. We used a search depth of 2:

max node → chance node → max node → chance node → evaluation function

As the depth increases, the computation time does as well. The learned value function was integrated in the existing Expectimax class. At the leaf nodes, the N-tuple network evaluates the resulting board. Expectimax then uses these values to determine which initial move has the highest expected outcome.

```
# Learned value function or the previous heuristic can be used.
def evaluate(self, state):
    if self.network is not None:
        return self.network.value(state, validate=False)
    empty = len(empty_cells(state))
    max_tile = np.max(state)
    return (100 * empty) + max_tile
```

Figure 9. The evaluation function used at Expectimax leaf nodes.

Evaluating the Agent

To evaluate our approach, we compared three agents in the same 2048 environment: the Random agent, the heuristic Expectimax agent, and the Expectimax + RL agent. The two Expectimax agents use the same depth-2 search; the only difference is how their leaf states are evaluated. The heuristic agent uses our hand-designed evaluation function, while the RL agent uses the trained N-tuple value function. Each agent played 30 games, and we recorded the average, median, and best score, highest tile reached, number of games reaching 2048, and average time per move.

Agent	Games	Avg	Median	Best	Best tile	Reached 2048	ms/move
Random	30	1,178	990	3,076	256	0/30	0.0
Expectimax (heuristic)	30	14,724	15,698	27,308	2,048	5/30	8.8
Expectimax + RL	30	62,244	66,646	120,744	8,192	30/30	12.9

Table 1. Results from 30 games per agent.

The results show that the approach works. The RL agent achieved an average score more than four times higher than the heuristic Expectimax agent (62,244 vs. 14,724) and reached the 2048 tile in all 30 games, compared with 5/30 for the heuristic agent and 0/30 for Random. Its best game reached the 8192 tile with a score of 120,744. This improvement comes with a small performance cost: the RL agent takes about 4 ms more per move, which is still fast enough for real-time gameplay. Since the heuristic and RL agents use the exact same search strategy, the performance difference comes from the value function: the N-tuple network trained with TD learning provides the stronger board evaluations that make the RL agent significantly more effective.

UI: The Pygame Interface

To watch the agents play, we built a Pygame interface based on the original 2048 game. The UI is separated from the game logic: the renderer is responsible only for displaying the game, while the game environment handles the 2048 rules and state, and the AI agents independently decide which actions to take.

The main menu offers the three agents and the comparison screen, selected with the keys 1 to 4 or by clicking the buttons:

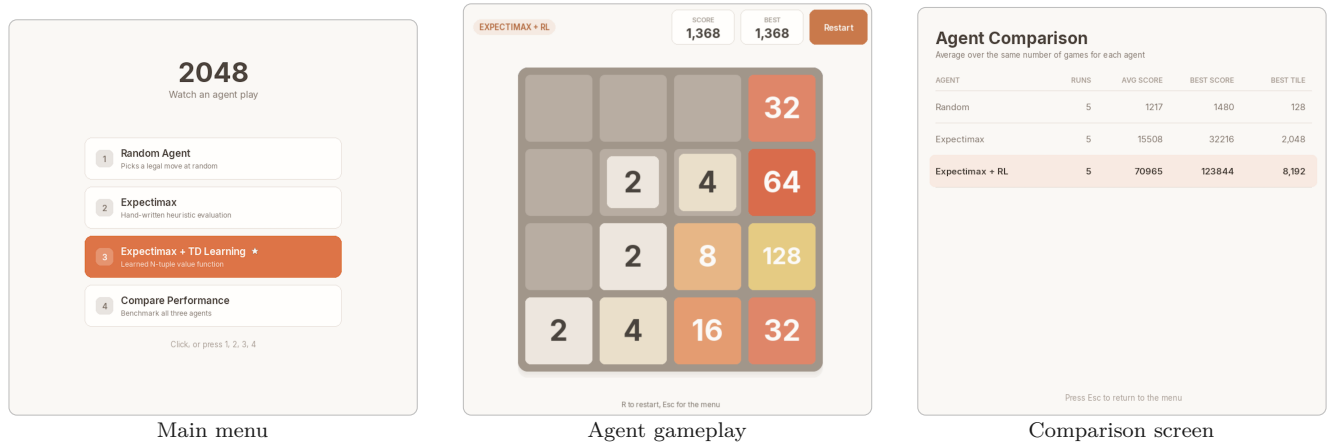


Figure 10. The Pygame interface.

Picking an agent starts a game and the agent plays by itself. Tiles slide and merge with animation, and the score updates in the header. Press **R** to restart the game, or **Esc** to return to the menu. When the game ends, an overlay shows the final score and the best tile reached.

The agent comparison screen runs a short benchmark of 5 games per agent and shows a live table with the average score, best score and highest tile for each.

Troubleshooting

Problem 1: ModuleNotFoundError: No module named 'agents'

While running the training, if the command is `python3 training/td_learning.py`, this will produce an error because the script is being executed from inside a subdirectory or Python is not using the project root as its working directory.

Solution: Run `python3 -m training.td_learning` from the root directory of the project.

Problem 2: ModuleNotFoundError: No module named 'pygame'

This error is when the program fails to start and pygame is unable to launch. This happens when the required Python dependencies have not been installed in the current Python environment.

Solution: From the project root, install the dependencies listed in `requirements.txt` using `pip install -r requirements.txt`, then run the project again with `python3 main.py`.

Conclusion

This guide presented the main techniques we used to build our 2048 AI agent. We first implemented the 2048 environment, then trained an N-tuple network value function using Temporal Difference learning and self-play and integrated the learned value function with Expectimax search to allow the agent to make decisions during gameplay. The main takeaway is that the different components work together as a complete reinforcement learning pipeline. This approach allowed us to replace a manually designed heuristic with a value function learned directly from gameplay experience.